

interface

Șapte exerciții — legi lucruri care n-au nicio rudă comună · Java · Adrian

Unde lucrezi: repo-ul `java-abstractizare`, peste ce ai făcut la `abstract` — ierarhia de figuri și cea de orar, amândouă cu clasa de sus abstractă.

Nou, ai voie: `interface`, `implements`, `instanceof` **cu numele unei interfețe**, `Collections.sort`.

Încă NU folosim: metode default sau static *în interfețe*, static în general, generice proprii, `enum`, `record`, fișiere, `try/catch`, `stream-uri`.

Cum se lucrează: fiecare exercițiu îți cere să **construiești** ceva. Rândul **Gata când** spune cum verifici; se verifică prin **rulare**.

Partea 1 — un tip comun peste ierarhii diferite

1. Un singur nume pentru „știi să mă copiezi”

nivel 1

Acum aceeași capacitate apare de trei ori, sub două nume: `Figura.duplicare()`, `Interval.duplicate()` și `Orar.duplicate()`. Iar `Orar` nu e nici figură, nici interval.

Scrie o interfață cu o singură metodă pentru capacitatea asta și fă toate trei s-o implementeze, **sub același nume de metodă**.

Apoi scrie, într-o clasă din afara celor două pachete, o metodă care primește o listă de astfel de obiecte și întoarce lista copiilor. În `App`: pune în **aceeași listă** un `Cerc`, o `OraCurs` și un `Orar`, cheam-o, și afișează originalele și copiile.

Regula exercițiului: metoda care copiază n-are voie cu `instanceof`, cu cast sau cu `if` după tip.

Gata când: lista din `App` e declarată o singură dată, ține obiecte din ambele pachete, iar programul afișează 3 originale și 3 copii.

De reținut: cu `extends` n-ai fi putut face lista asta — un `Orar` n-are ce moșteni de la o `Figura`.

2. O capacitate pe care n-o au toți

nivel 1

`Cerc`, `Dreptunghi` și `Triunghi` au arie. `Punct` și `Linie` nu au.

Scrie o interfață pentru „figură care are arie” și fă doar cele trei s-o implementeze. Apoi scrie în `Desen` o metodă care adună ariile figurilor care au arie și le trece cu vederea pe celelalte.

Regula exercițiului: ai voie `instanceof` cu numele interfeței, dar nu cu numele unei clase.

Gata când: adaugi un `Romb` cu arie, îl pui în desen, și e numărat **fără nicio modificare** în `Desen`.

De reținut: întrebarea „ce poți face” rezistă la clase noi. Întrebarea „ce ești” te obligă să treci pe la fiecare `instanceof` și să-l completezi.

3. O clasă, două contracte

nivel 1

Un `Cerc` le are pe amândouă: se copiază **și** are arie.

Scrie două metode separate — una care primește un obiect „care se copiază”, alta care primește un obiect „care are arie” — și cheamă-le pe **același** `Cerc`. Apoi încearcă să dai aceluiași `Punct` la amândouă și vezi care dintre ele îl primește.

Gata când: același obiect `Cerc` intră în ambele metode, iar tu poți scrie într-un rând de ce `Punct`-ul intră doar în una.

De reținut: părintele e unul singur, contractele sunt câte vrei. `Cerc extends Figura implements ...` e o linie normală.

Partea 2 — implementezi un contract scris de altcineva

4. Orarul se sortează singur

mai greu

Java are deja o interfață pentru „obiect care se poate compara cu altul de același fel”, iar `Collections.sort` știe să lucreze cu orice o implementează.

Fă `Interval` să o implementeze, comparând după ora de început. Apoi, în `Orar`, scrie o metodă care sortează intervalele și afișează orarul în ordine cronologică.

În `App`: adaugă intervalele **intenționat în dezordine** (examenul primul, pauza înainte de curs) și afișează orarul înainte și după sortare.

Gata când: orarul afișat după sortare e în ordinea orelor, oricum le-ai fi adăugat; iar tu n-ai scris nicăieri un algoritm de sortare.

De reținut: aici n-ai inventat tu contractul — l-ai respectat pe al altcuiva, ca să-ți poți folosi codul lui. Asta e jumătate din programarea cu biblioteci.

5. Eticheta, fără moștenire

mai greu

Acum `Eticheta` moștenește `Dreptunghi` **și în plus** ține un câmp `Dreptunghi` — aceleași două puncte, păstrate în două locuri.

Scrie o a doua variantă de `Eticheta`, care **nu** moștenește `Dreptunghi`, ci doar ține unul pe dinăuntru și implementează contractele de care are nevoie ca să stea într-un `Desen`.

Ca să încapă, `Desen` nu mai poate cere o clasă anume pentru elementele lui. Decide tu ce cere în loc.

Gata când: ambele variante stau în același `Desen` și se afișează; iar pentru varianta nouă mutarea chiar mută eticheta — dovedește-o afișând coordonatele înainte și după.

De reținut: numără câmpurile fiecărei variante. Una ține perechea de puncte o dată, cealaltă de două ori — și un setter le schimbă pe rând, nu pe amândouă.

6. Un intrus în listă

provocare

Scrie o clasă care nu e nici figură, nici interval, nici orar — de exemplu o **Notiță** cu un titlu și un text — și fă-o să se poată copia, după același contract ca la exercițiul 1. Pune-o în pachetul ei.

Adaug-o în aceeași listă cu `Cerc`, `OraCurs` și `Orar`, și treci lista prin aceeași metodă de copiere.

Gata când: notița e copiată corect (modifici textul copiei, originalul rămâne neschimbat), iar metoda care copiază **n-a fost atinsă deloc** — verifică cu `git diff` că fișierul ei nu apare între modificări.

De reținut: asta e testul adevărat al unei interfețe — un tip la care nu te-ai gândit când ai scris-o întră fără să schimbi nimic.

7. Figura : clasă abstractă sau interfață?

provocare

Ai acum amândouă uneltele. Uită-te la `Figura` așa cum arată și răspunde în scris, în cinci-șase rânduri: ar trebui să rămână clasă abstractă sau ar trebui să devină interfață?

Argumentează pornind de la două întrebări concrete:

- Ce **stare** au în comun toate figurile, și cine ar ține-o dacă `Figura` ar fi interfață?
- Există cod scris o singură dată în `Figura` și folosit de toate subclasele? Ce s-ar întâmpla cu el?

Apoi implementează varianta pe care ai ales-o și rulează programul întreg.

Gata când: programul rulează la fel ca înainte cu varianta ta, iar motivarea scrisă se potrivește cu ce a ieșit în cod — inclusiv cu ce a trebuit să muți sau să dublezi.

De reținut: cele două nu se exclud. Cea mai obișnuită soluție e o clasă abstractă **plus** interfețe: familia vine din moștenire, capacitățile din contracte.